As bajo la manga

Sistema Web de Reservas para Barbería

Presentación 2 — Diseño de la solución · 12/10/2026

SAL-301 Algoritmos y Programación · UTEPSA · Ing. Gloria M. Gonzales Fernandez

Del análisis al diseño

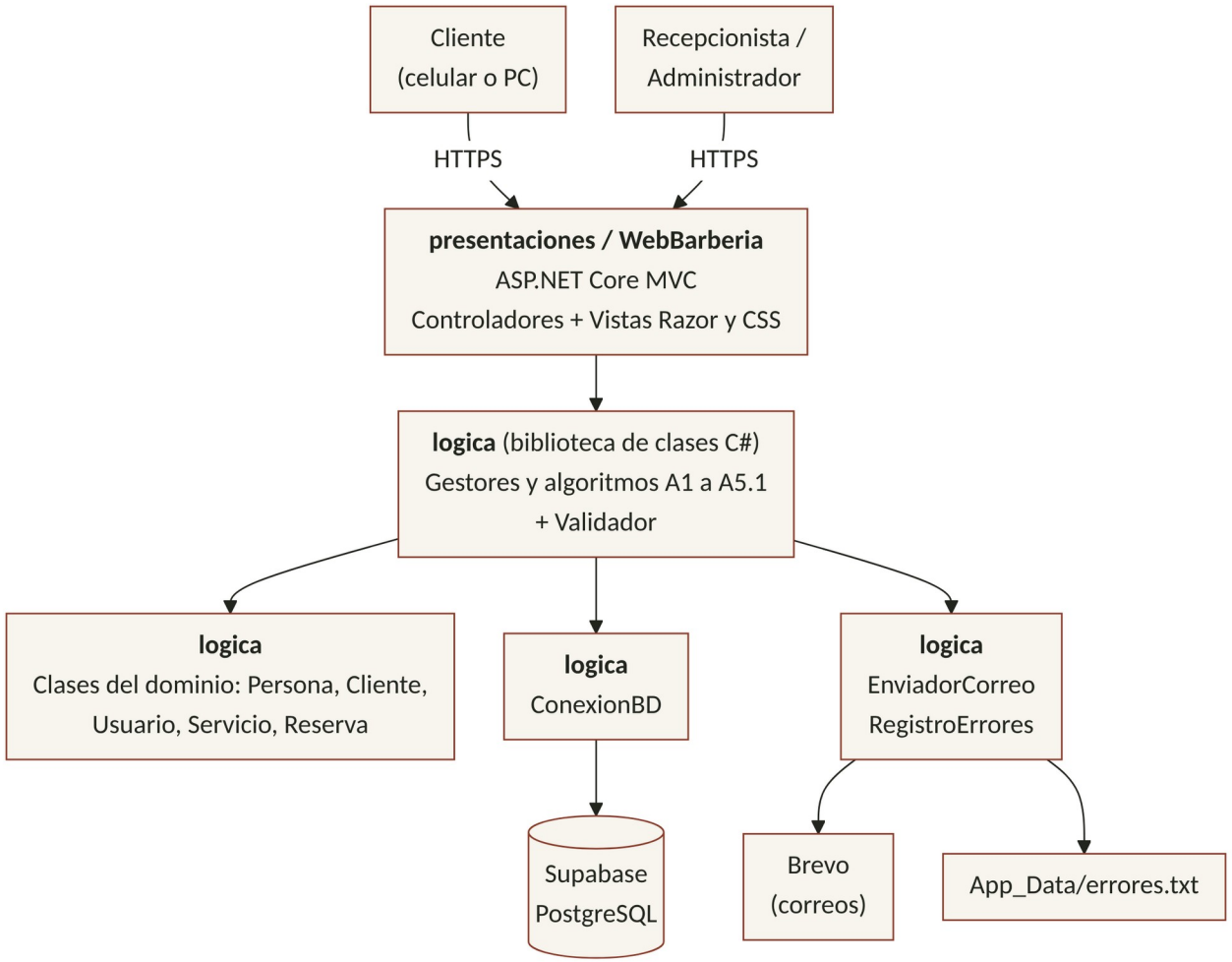
23	12	6	23
requerimientos funcionales	no funcionales	diagramas UML verificados con el código	pruebas automáticas que pasan

Problema: las citas se coordinan por WhatsApp, llamadas o un cuaderno; hay choques de horario y citas perdidas.

Solución: web en C# (ASP.NET Core MVC): el cliente reserva en 4 pasos y el personal gestiona la agenda en un panel con inicio de sesión.

Regla central (RN-06): [horaInicio, horaFin) choca si horaInicio < horaFinExistente y horaFin > horaInicioExistente.

Arquitectura: 2 proyectos, como en clase



1 **logica (biblioteca)**

Clases del dominio, gestores con los algoritmos, validaciones, seguridad, datos, correo y la excepción propia.

2 **WebBarberia (MVC)**

Controladores, vistas Razor y CSS. Usa la capa logica.

3 **Logica.Pruebas (xUnit)**

23 pruebas de los algoritmos y las reglas, sin base de datos.

Diagramas UML de clases

Los nombres de clases, atributos y métodos son los mismos del código:
el script que dibuja el UML los verifica contra los archivos .cs

Criterio de diseño orientado a objetos

- Atributos privados

Ej.: Reserva tiene -fecha, -horaInicio, -estado. Nadie los cambia desde afuera.

( ) Constructor que valida

Servicio exige duración múltiplo de 30 y precio > 0; Cliente valida teléfono y correo.

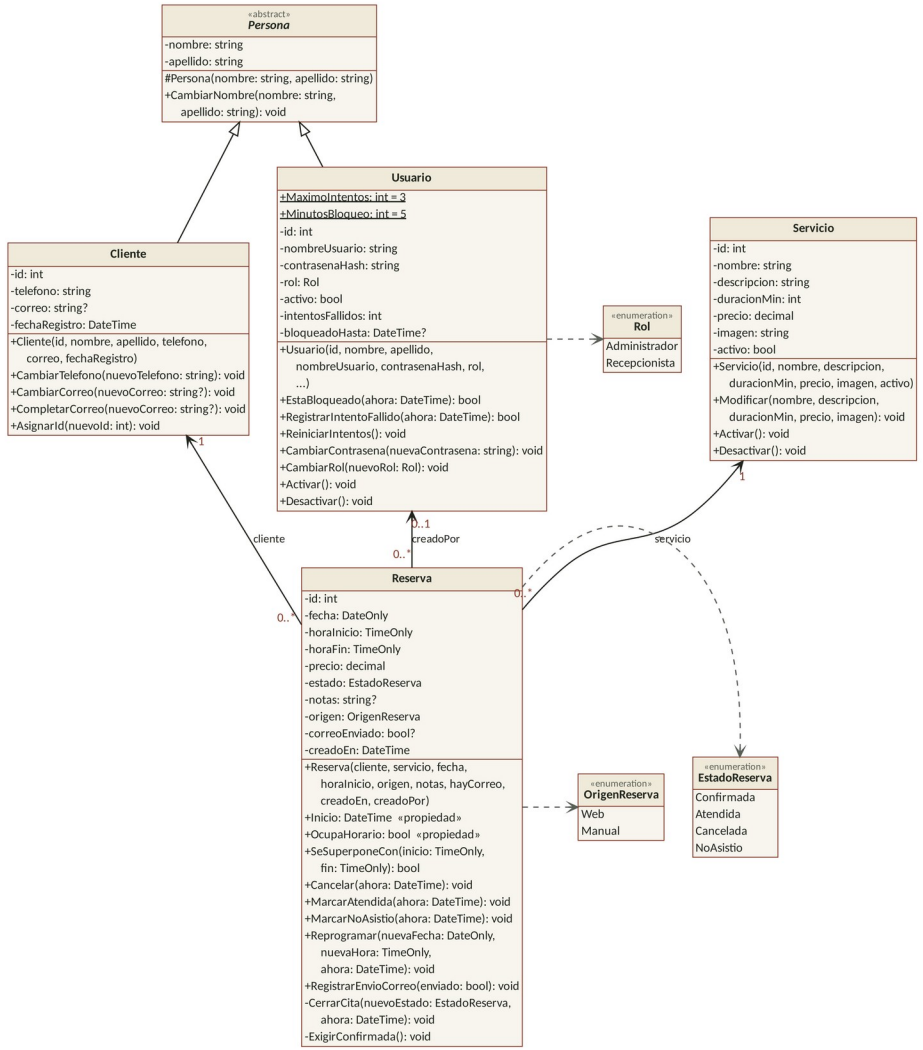
+ Métodos con reglas

Cancelar, MarcarAtendida, Reprogramar aplican RN-13 y RN-14; si no se cumplen, lanzan excepción.

▲ Herencia

Persona (abstracta) → Cliente y Usuario.
HorarioNoDisponibleException hereda de Exception.

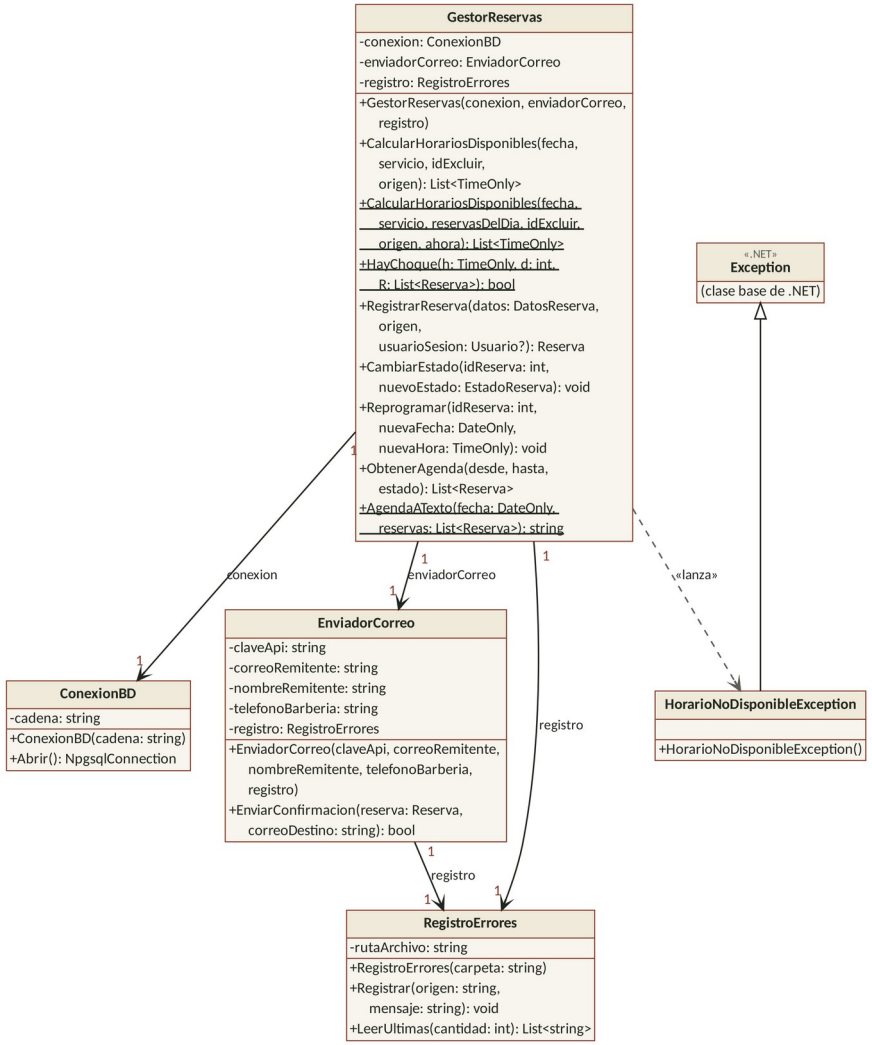
Modelo del dominio



Relación	Multiplicidad
Cliente, Usuario → Persona (herencia)	—
Reserva → Cliente	0..* a 1
Reserva → Servicio	0..* a 1
Reserva → Usuario (creadoPor)	0..* a 0..1
HorarioAtencion ◆ Turno	1 a 2
Reporte ◆ ConteoServicio	1 a 0..*

Las reservas web no tienen usuario (creadoPor vacío); el precio se copia del servicio al crearlas (RN-15).

GestorReservas: el corazón de la lógica



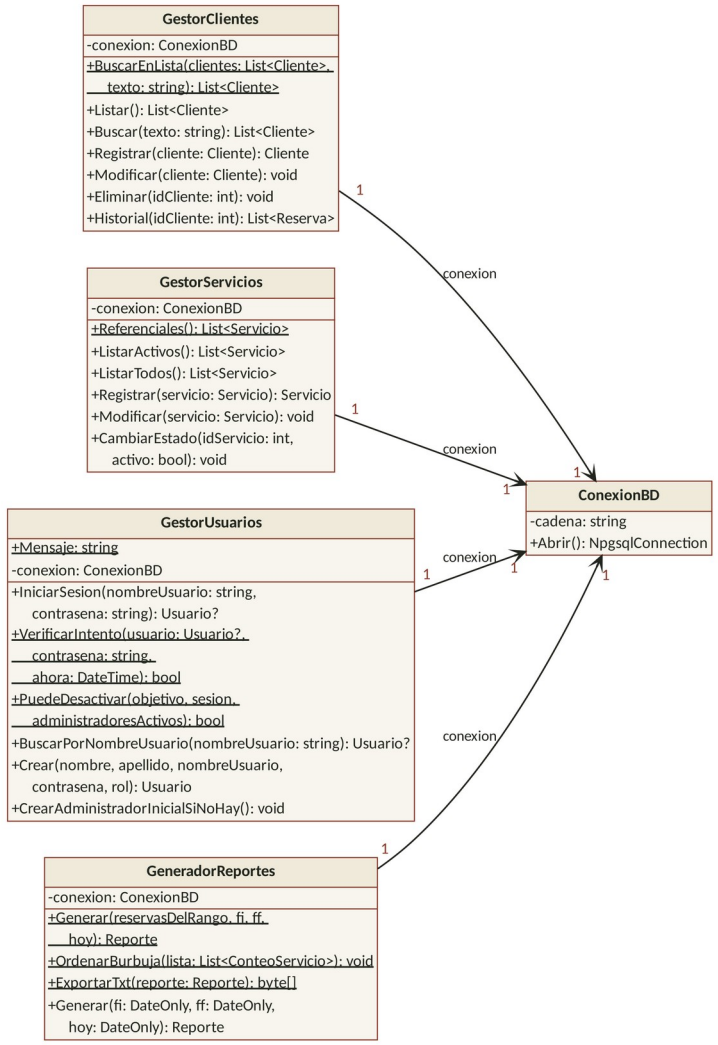
A1 CalcularHorariosDisponibles: bloques libres según la duración del servicio.

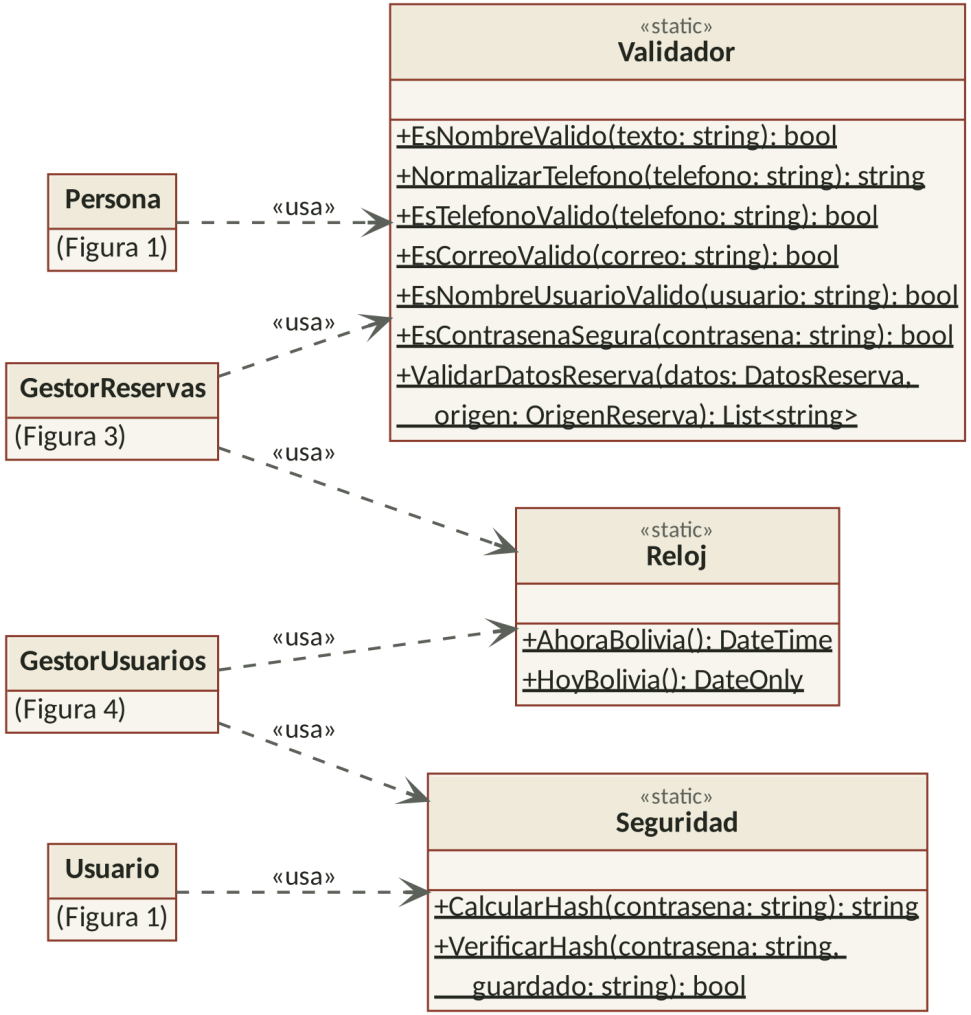
A1.1 HayChoque: aplica la fórmula de RN-06 a la lista de reservas del día.

A2 RegistrarReserva: valida, revalida el horario, guarda y envía el correo.

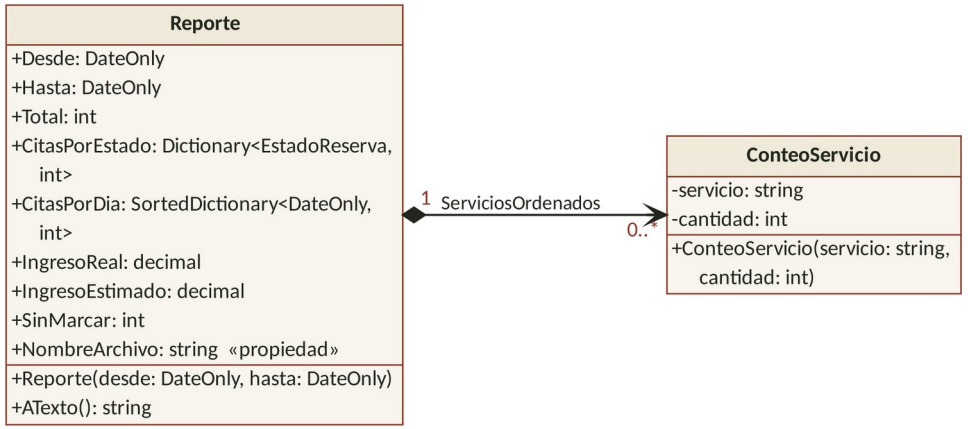
A4 CambiarEstado y Reprogramar: delegan las reglas a Reserva.

Gestores y clases utilitarias

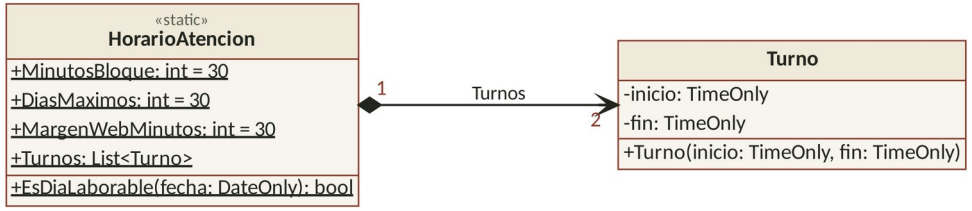


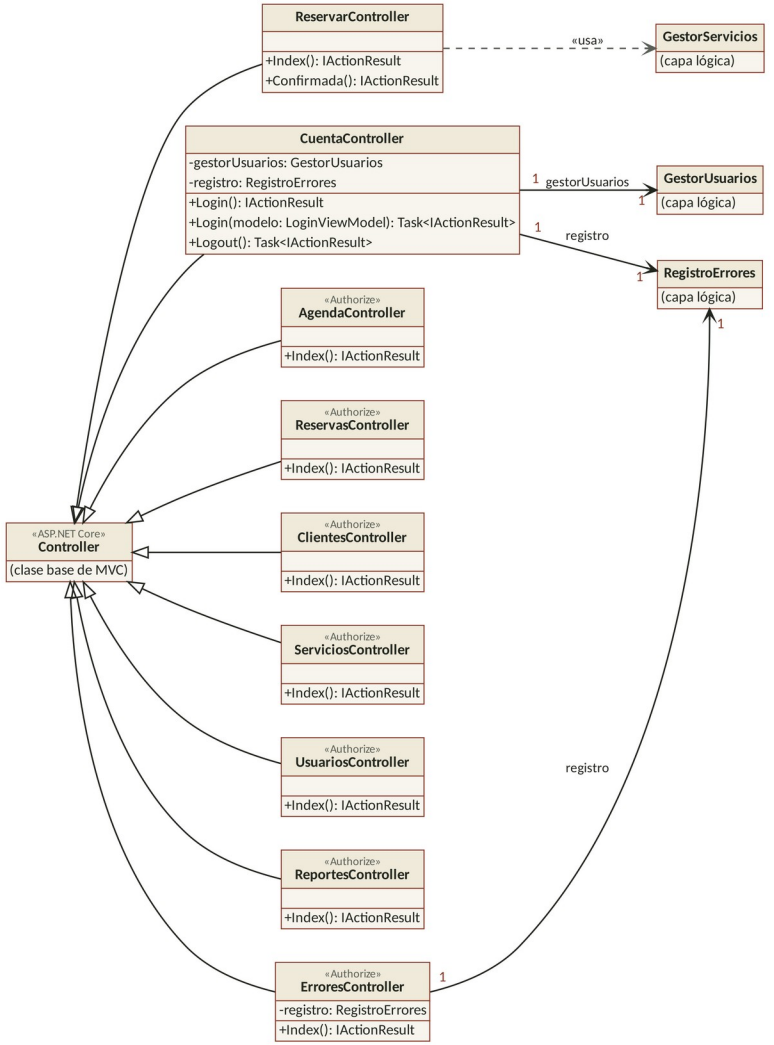


Horario, reporte y capa de presentación







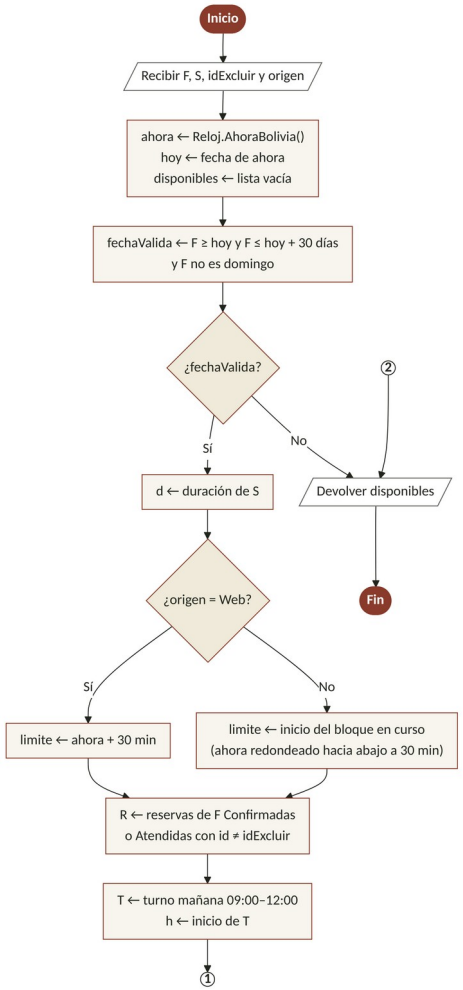


Algoritmo central A1

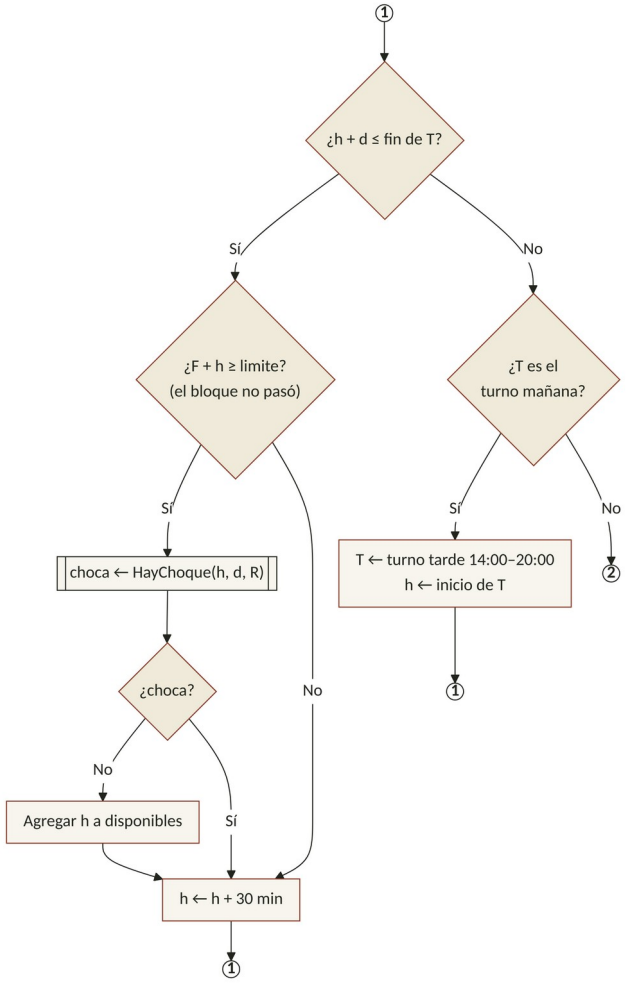
Calcular horarios disponibles según la duración del servicio

Complejidad O(b × r): b bloques del día (máx. 18) × r reservas del día

A1 — Diagrama de flujo



Parte 1: validación y preparación



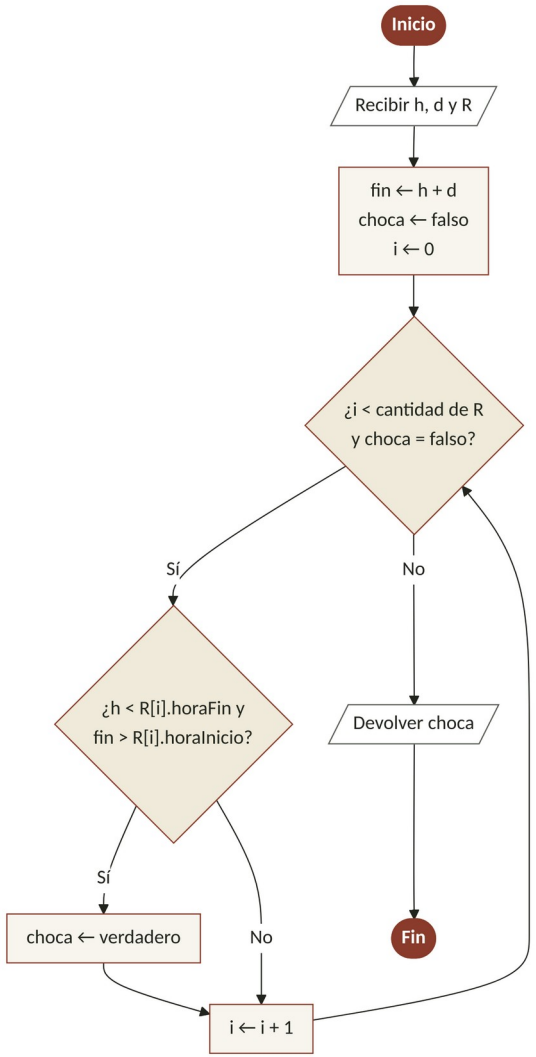
Parte 2: recorrido de los bloques

A1 — Pseudocódigo (CalcularHorariosDisponibles)

```
Funcion disponibles <- CalcularHorariosDisponibles(
         F, S, reservasDelDia, idExcluir, origen, ahora)
    Definir disponibles, R Como Lista
    Definir hoy Como Fecha
    Definir limite Como FechaHora
    Definir d Como Entero
    Definir h Como Hora
    Definir fechaValida, choca Como Logico
    disponibles <- ListaVacia()
    hoy <- FechaDe(ahora)
    fechaValida <- F >= hoy Y F <= hoy + 30 dias Y EsDiaLaborable(F)
    Si fechaValida Entonces
        d <- S.DuracionMin
        Si origen = Web Entonces
            limite <- ahora + 30 minutos
        SiNo
            limite <- InicioDelBloqueEnCurso(ahora)
        FinSi
```

```
R <- ListaVacia()
Para Cada r En reservasDelDia Hacer
    Si r.Fecha = F Y r.OcupaHorario Y r.Id <> idExcluir Entonces
        Agregar(R, r)
    FinSi
FinPara
Para Cada T En HorarioAtencion.Turnos Hacer
    h <- T.Inicio
    Mientras h + d <= T.Fin Hacer
        Si F + h >= limite Entonces
            choca <- HayChoque(h, d, R)
            Si NO choca Entonces
                Agregar(disponibles, h)
            FinSi
        FinSi
        h <- h + 30 minutos
    FinMientras
FinPara
    FinSi
FinFuncion
```

A1.1 — Subalgoritmo HayChoque




```
Funcion choca <- HayChoque(h, d, R)
    Definir fin Como Hora
    Definir i Como Entero
    fin <- h + d
    choca <- Falso
    i <- 0
    Mientras i < Longitud(R) Y NO choca Hacer
        Si h < R[i].HoraFin Y fin > R[i].HoraInicio Entonces
            choca <- Verdadero
        FinSi
        i <- i + 1
    FinMientras
FinFuncion
```

=

Intervalos semiabiertos [inicio, fin)

Las citas contiguas no chocan: 10:00–11:00 y 11:00–12:00 se permiten porque 11:00 no es menor que 11:00.

Prueba de escritorio de A1

h	h + d	¿HayChoque?	Resultado
09:00	10:00	No (10:00 no es mayor que 10:00)	Disponible
09:30 a 10:30	10:30 a 11:30	Sí, con 10:00–11:00	—
11:00	12:00	No (11:00 no es menor que 11:00)	Disponible
11:30	12:30	No entra en el turno: termina el Mientras	—
14:00	15:00	No	Disponible
14:30 a 16:00	15:30 a 17:00	Sí, con 15:00–16:30	—
16:30 a 19:00	17:30 a 20:00	No	6 disponibles

9

horarios disponibles
09:00, 11:00, 14:00, 16:30, 17:00, 17:30, 18:00, 18:30, 19:00

Datos: lunes 12/10/2026, Corte + barba (d = 60 min), origen Web, R = {10:00–11:00, 15:00–16:30}.

Estructura del proyecto C#

Solución .NET 10 · compila sin errores ni advertencias · 23 pruebas pasan

Organización de la solución

```
codigo/
├── BarberiaAsBajoLaManga.slnx
├── logica/
│   ├── Modelos/        Persona, Cliente, Usuario,
│   │                   Servicio, Reserva, Turno, ...
│   ├── Enumeraciones/  EstadoReserva, Rol, Origen...
│   ├── Gestores/       GestorReservas, GestorClientes,
│   │                   GestorServicios, GestorUsuarios,
│   │                   GeneradorReportes
│   ├── Utilidades/     Reloj, Validador, Seguridad,
│   │                   RegistroErrores
│   ├── Datos/          ConexionBD (Npgsql)
│   ├── Integraciones/  EnviadorCorreo (Brevo)
│   └── Excepciones/    HorarioNoDisponibleException
├── presentaciones/WebBarberia/   ASP.NET Core MVC
├── pruebas/Logica.Pruebas/       23 pruebas xUnit
└── basedatos/   01_esquema.sql, 02_datos_iniciales.sql
```

1 Menú principal

_LayoutPanel: Agenda, Reservas, Clientes; Servicios, Usuarios, Reportes y Errores solo para el Administrador.

2 Base de datos

4 tablas con CHECK y la restricción EXCLUDE anti-choques (error 23P01 → HorarioNoDisponibleException).

3 Archivos de texto

errores.txt (escribir y leer) y exportación de agenda y reportes a .txt.

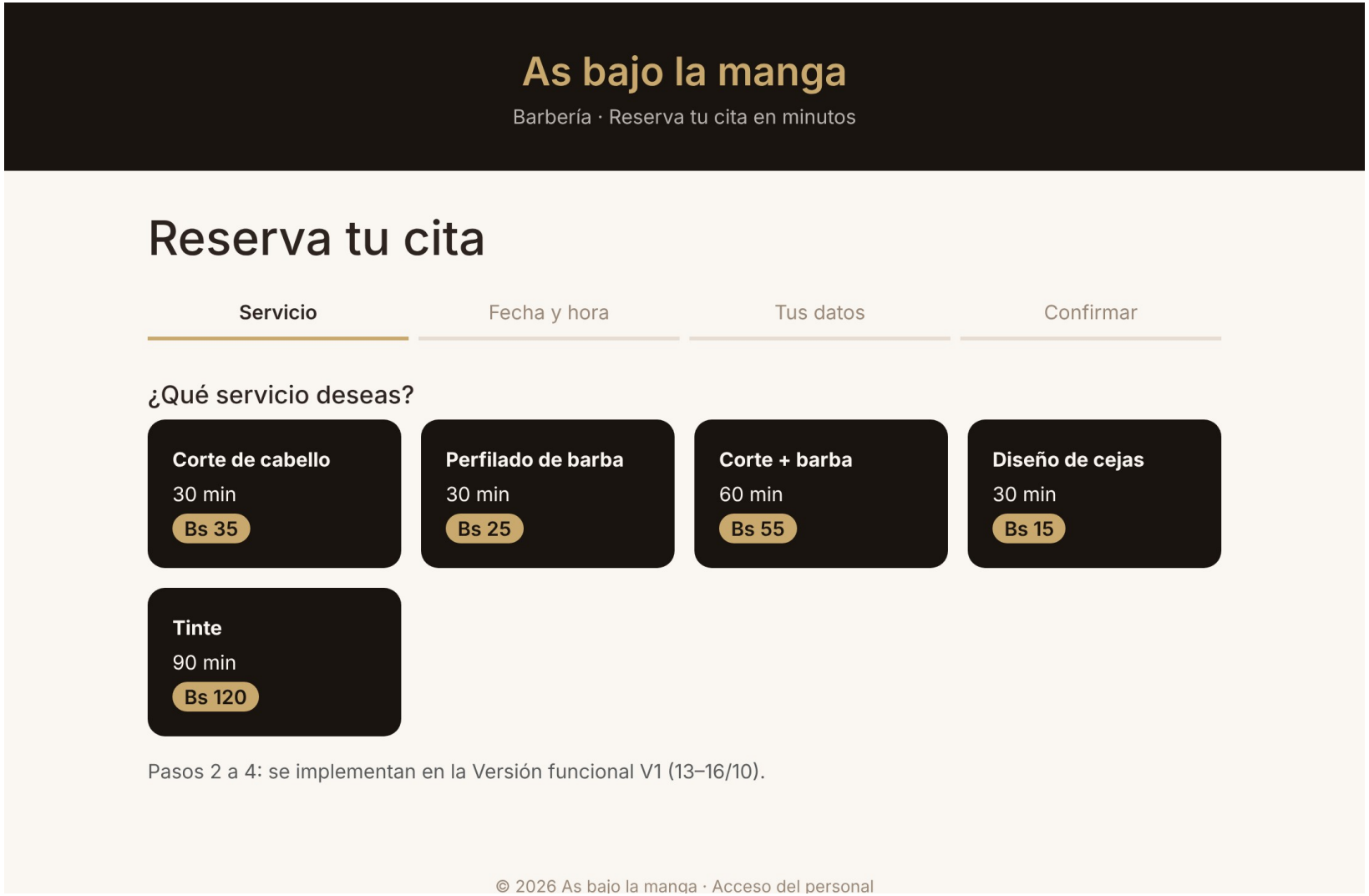
Pruebas automáticas de la lógica

23 / 23

pruebas xUnit pasan
(dotnet test)

Prueba	Verifica
PruebaDeEscritorio_DevuelveLosNueveHorarios	A1 con la prueba de escritorio
HayChoque_CitasContiguasNoChocan	A1.1 y RN-06
ServicioQueCruzaElDescanso_NoSeOfrece	RN-05
FechasNoValidas_NoTienenHorarios (3 casos)	RN-08
Hoy_LaWebExigeTreintaMinutos…	RN-09
Burbuja_OrdenaDeMayorAMenor	A5.1
Reserva_NoSeCancelaSiYaEmpezo…	RN-13, RN-14
InicioDeSesion_TresIntentosFallidos…	A3 y RN-18
BusquedaLineal_EncuentraPorNombreOTelefono	Búsqueda O(n)

Estructura inicial funcionando



Paso 1
Servicio: tarjetas con duración y precio.

Paso 2
Fecha y hora: bloques ocupados tachados (A1).

Paso 3
Datos del cliente, validados.

Paso 4
Resumen y «¡Cita confirmada!» + correo.

Del algoritmo al código y próximos pasos

Algoritmo	Método en C#	Clase
A1 Horarios disponibles	CalcularHorariosDisponibles	GestorReservas
A1.1 HayChoque	HayChoque	GestorReservas
A2 Registrar reserva	RegistrarReserva	GestorReservas
A3 Iniciar sesión	IniciarSesion, VerificarIntento	GestorUsuarios
A4 Estado / reprogramar	CambiarEstado, Reprogramar	GestorReservas, Reserva
A5 Reporte	Generar	GeneradorReportes
A5.1 Burbuja	OrdenarBurbuja	GeneradorReportes

Implementación 13–16/10

- Consultas a Supabase con parámetros y la transacción de A2
- Pasos 2 a 4 de la reserva web
- Panel: agenda con exportación .txt, reserva manual y cambios de estado
- CRUD de clientes y registro de errores

Gracias

Preguntas